

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192512132
Name	G DHANUSH

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I **G DHANUSH (192512132)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G DHANUSH
(192512132)

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. End-to-End System Design

Problem Statement

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations. The system should process large volumes of social media posts, identify useful information, analyze user interests, and provide personalized recommendations.

Objectives

- ❖ Collect real-time social media data.
- ❖ Process large-scale data using PySpark.
- ❖ Clean and transform the collected data using RDD operations.
- ❖ Analyze user interests and trends.
- ❖ Generate personalized recommendations using an intelligent agent.
- ❖ Provide recommendations with low processing delay.

Data Flow :

Step 1 – Data Collection:

The system collects posts, comments, hashtags, likes, and other relevant social media information.

Step 2 – Data Ingestion:

The incoming information is transferred to the distributed processing system.

Step 3 – RDD Creation:

PySpark creates RDDs containing the collected social media data.

Step 4 – Data Preprocessing:

Unwanted data is removed and useful information is extracted.

Step 5 – Feature Extraction:

Important features such as keywords, hashtags, topics, and user interests are identified.

Step 6 – AI Analysis:

The AI model analyzes the extracted features to identify trends and user preferences.

Step 7 – Agent Decision:

The intelligent agent uses the analysis to determine appropriate recommendations.

Step 8 – Recommendation:

The final recommendations are provided to the user.

PySpark RDD Operations :

Example operations:

Python

Run

```
data = sc.textFile("social_media_data.txt")

clean_data = data.filter(lambda x: len(x.strip()) > 0)

words = clean_data.flatMap(lambda x: x.split())

word_pairs = words.map(lambda x: (x.lower(), 1))

word_count = word_pairs.reduceByKey(lambda a, b: a + b)

result = word_count.collect()
```

Operations used:

- `filter()` – removes empty or unwanted records.
- `flatMap()` – separates posts into individual words.
- `map()` – converts words into key-value pairs.
- `reduceByKey()` – calculates the frequency of each word.
- `collect()` – retrieves the processed result.

Intelligent Agent

The recommendation agent observes user activity and processed social-media information.

Percepts: Posts, hashtags, likes, comments, and user preferences.

Goal: Generate relevant recommendations.

Actions: Recommend content, topics, products, or services.

Environment: Social media platform and its users.

Scalability

PySpark divides the large dataset into partitions and processes them across multiple worker nodes. Therefore, increasing the number of nodes can increase the processing capacity.

Fault Tolerance

RDDs are fault tolerant because lost partitions can be recomputed using their lineage information.

Result

The system analyzes large-scale social media data and produces personalized recommendations efficiently using distributed processing.

2. Distributed Intelligent Agent System

Problem Statement

Develop a **multi-agent AI system** in which multiple intelligent agents collaborate to solve a large-scale optimization problem while PySpark performs distributed data processing.

Example Problem

A suitable example is **delivery-route optimization** for a large number of delivery vehicles.

The objective is to determine efficient routes while considering:

- ✓ Distance
- ✓ Delivery time
- ✓ Traffic conditions
- ✓ Vehicle availability
- ✓ Fuel/energy consumption

Architecture :

Traffic Cameras

Traffic Sensors

GPS Data



Real-Time Data Stream



PySpark



RDD Processing



Traffic Feature Extraction



Intelligent Agents

Traffic Agent

Signal Agent

Emergency Agent

Prediction Agent



Decision Engine



↓ ↓ ↓

Signal Route Alert
Control Suggestion System

Agent Roles

1. Route Agent

Finds possible routes between source and destination.

2. Traffic Agent

Analyzes traffic-related information and identifies congested routes.

3. Cost Agent

Evaluates distance, time, and resource consumption.

4. Coordination Agent

Combines the outputs of the other agents and selects the best solution.

PySpark Processing

Large route datasets can be distributed among multiple worker nodes.

Example:

Python

Run

```
routes = sc.textFile("routes.txt")
```

```
valid_routes = routes.filter(
```

```
    lambda x: x.strip() != ""
```

```
)
```



```
route_data = valid_routes.map(

    lambda x: x.split(",")

)

cost_data = route_data.map(

    lambda x: (x[0], float(x[3]))

)

best_routes = cost_data.reduceByKey(

    lambda a, b: min(a, b)

)
```

Collaboration Process

1. PySpark distributes the input data.
2. Route Agent calculates possible routes.
3. Traffic Agent evaluates traffic conditions.
4. Cost Agent calculates route cost.
5. Coordination Agent compares the results.
6. The best route is selected.

Advantages

- Parallel processing
- Reduced processing time
- Agent specialization
- Scalability

Result

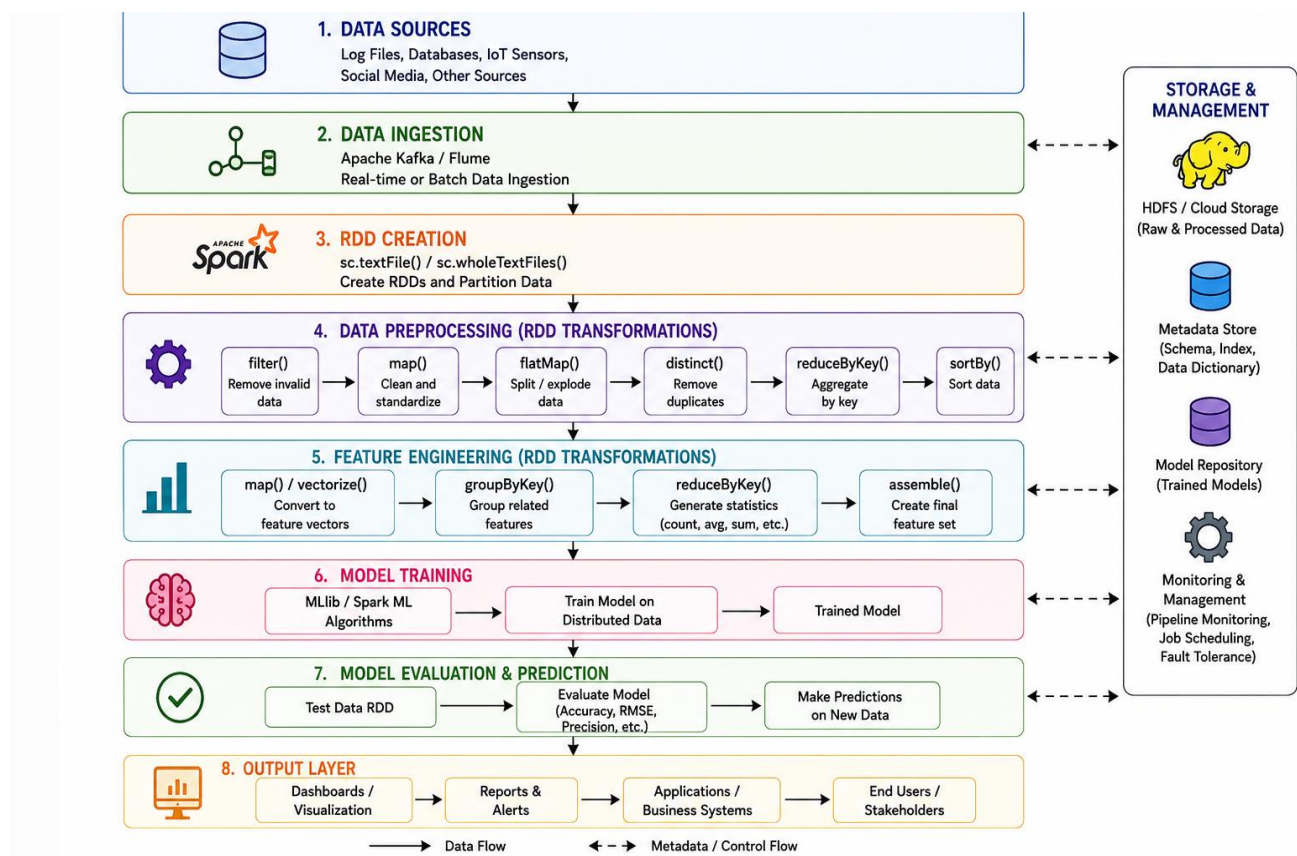
The multi-agent architecture solves large optimization problems by dividing the problem into smaller tasks and allowing specialized agents to collaborate.

3. Big Data AI Pipeline Design

Problem Statement

Design a complete distributed AI pipeline using **RDD transformations** for preprocessing, feature engineering, and model training while maintaining scalability and fault tolerance.

Pipeline Architecture



Step 1 – Data Ingestion

Large datasets are loaded into Spark.

Python

Run

```
data = sc.textFile("dataset.csv")
```

Step 2 – Data Cleaning

Invalid and empty records are removed.

Python

Run

```
cleaned = data.filter(lambda x: x.strip() != "")
```

Step 3 – Transformation

The data is divided into useful fields.

Python

Run

```
records = cleaned.map(lambda x: x.split(","))
```

Step 4 – Feature Engineering

Relevant features are extracted from the dataset.

Python

Run

```
features = records.map(  
    lambda x: (x[0], float(x[1]), float(x[2]))  
)
```

Step 5 – Aggregation

RDD operations can aggregate information.

Python

Run

```
result = features.reduceByKey(  
    lambda a, b: a + b
```

)

RDD Transformations

Important transformations include:

Operation	Purpose
<code>map()</code>	Transform each record
<code>filter()</code>	Remove unwanted records
<code>flatMap()</code>	Generate multiple outputs from one input
<code>reduceByKey()</code>	Aggregate key-value data
<code>groupByKey()</code>	Group records by key
<code>distinct()</code>	Remove duplicate records

RDD Actions

Examples include:

- `collect()`
- `count()`
- `first()`
- `take()`

Fault Tolerance

If a partition is lost, Spark can recompute it from the RDD lineage instead of restarting the entire process.

Result :

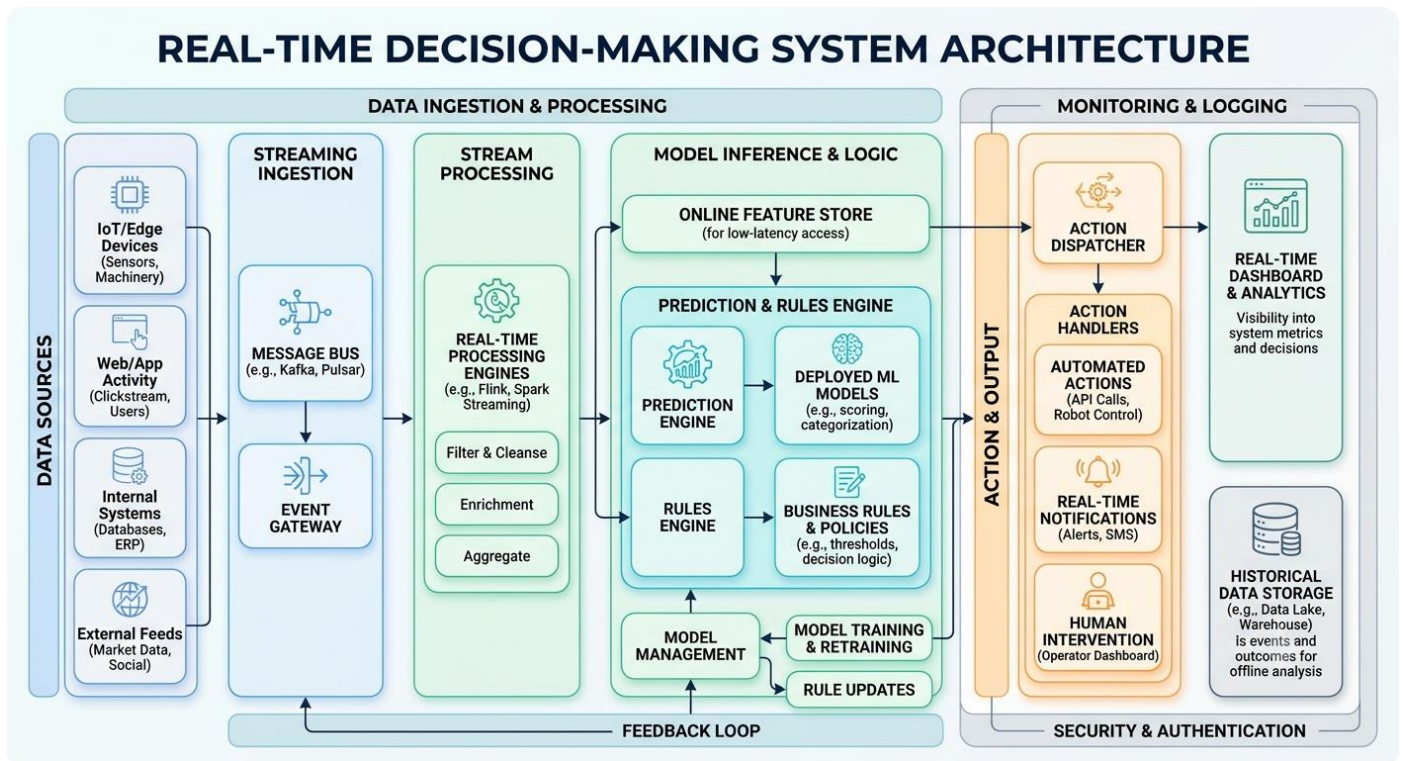
The pipeline provides scalable preprocessing, feature extraction, and distributed AI processing while supporting fault tolerance.

4. Real-Time Decision-Making System

Problem Statement

Construct an AI-based system that combines **RDD-based streaming data processing** with **intelligent agents** for real-time decision-making. A suitable application from the assignment is **smart traffic management**.

System Architecture



Input Data

The system can receive :

- ✓ Vehicle count
- ✓ Vehicle speed

- ✓ Traffic density
- ✓ Accident information
- ✓ Emergency vehicle information
- ✓ Road conditions

RDD Processing

Python

Run

```
traffic = sc.textFile("traffic_stream")
```

```
valid = traffic.filter(
```

```
    lambda x: x.strip() != ""
```

```
)
```

```
records = valid.map(
```

```
    lambda x: x.split(",")
```

```
)
```

```
density = records.map(
```

```
    lambda x: (x[0], int(x[1]))
```

```
)
```

```
total = density.reduceByKey(
```

```
    lambda a, b: a + b
```

```
)
```

Intelligent Agents

Traffic Monitoring Agent

Monitors traffic density.

Emergency Agent

Identifies accidents and emergency vehicles.

Decision Agent

Uses the processed information to decide whether traffic signals should be modified.

Decision Process

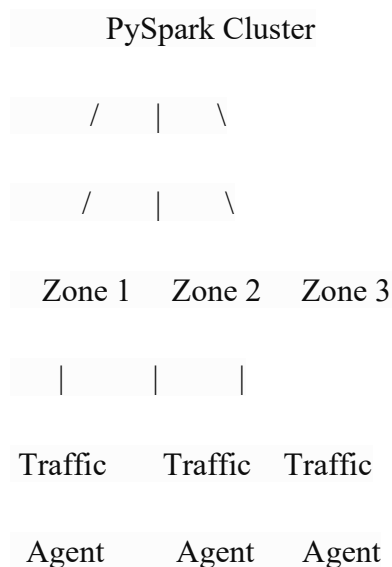
- I. Traffic Density High
- II. Traffic Agent detects congestion
- III. Decision Agent analyzes situation
- IV. Increase green-light duration
- V. Traffic congestion reduced

Real-Time Advantage

The system continuously processes incoming information and provides decisions without requiring the entire dataset to be processed at once.

Scalability

Different traffic regions can be processed in parallel.



Result :

The proposed system can detect traffic conditions and make intelligent decisions quickly using distributed processing and intelligent agents.

5. Cloud-Based Distributed AI Application

Problem Statement

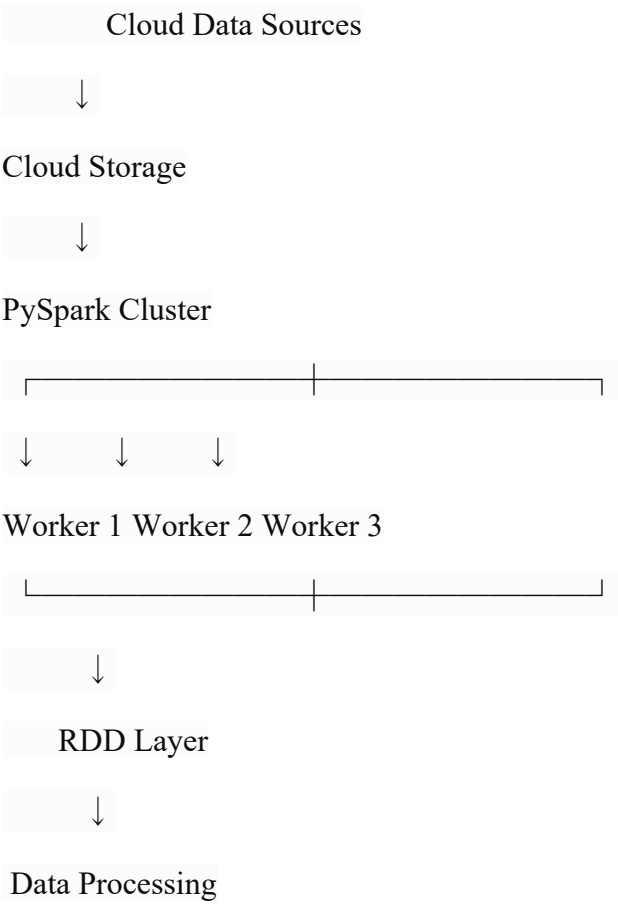
Design a **cloud-enabled AI system using PySpark and intelligent-agent architecture** for large-scale data analytics, dynamic resource allocation, and adaptive learning.

Proposed Application

A suitable design is a **Cloud-Based Intelligent Resource Management System**.

The system monitors cloud workloads and dynamically allocates computing resources according to demand.

Architecture





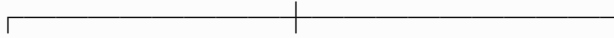
Feature Engineering



AI/ML Model

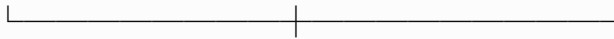


Intelligent Agents



Resource Learning Monitoring

Agent Agent Agent



Decision Engine



Cloud Application

Data Processing

PySpark processes large-scale cloud monitoring data.

Python

Run

```
logs = sc.textFile("cloud_logs.txt")
```

```
clean_logs = logs.filter(
```

```
    lambda x: x.strip() != ""
```

```
)
```

```
records = clean_logs.map(
```

```
    lambda x: x.split(",")
```

```
)
```

```
usage = records.map(

    lambda x: (x[0], float(x[2])))

)

average_usage = usage.groupByKey().mapValues(

    lambda values: sum(values) / len(values)

)
```

Intelligent Agents

1. Monitoring Agent

Collects CPU, memory, storage, and network usage.

2. Prediction Agent

Analyzes historical information and predicts future resource requirements.

3. Resource Allocation Agent

Determines how many resources are required.

4. Learning Agent

Learns from previous resource-allocation decisions and improves future decisions.

Dynamic Resource Allocation

Example:

- i. Low Workload
- ii. Reduce unnecessary resources
- iii. Save computing resources
- iv. High Workload
- v. Increase resources

- vi. Maintain performance

Adaptive Learning

The learning agent monitors previous decisions and their results.

For example:

- Observe workload.
- Allocate resources
- Monitor performance.
- Measure the result.
- Learn from the result.
- Improve the next allocation decision.

Fault Tolerance

Distributed processing allows failed tasks to be recomputed. The system can continue operating even if an individual worker experiences a failure

Result :

The proposed cloud-based AI system provides

- ✓ Large-scale data analytics
- ✓ Distributed processing
- ✓ Dynamic resource allocation
- ✓ Intelligent decision-making
- ✓ Adaptive learning
- ✓ Scalability
- ✓ Fault tolerance